EE4080 Final Year Interim Report

Manipulation of Deformable Linear Objects Using Graphical Neural Networks

School of Electrical & Electronic Engineering

Submitted by: Arpith Koshy

Matriculation Number: U2021054D

Supervisor: A/P Cheah Chien Chern

Mentor: A/P Lee Peng Hin

Table of Contents

Introduction 3

Background Information 3

Purpose 3

Scope 3

Objectives 4

1) Database creation 4

2) Image Preprocessing 4

3) GNN Model Development 4

4) Training and Validation 4

5) Evaluation and Validation 4

6) Optimization for Real-time Inference: 4

7) Robot Interaction Testing: 4

Timeline 5

Purpose of Report 5

Work Conducted 5

Image Processing and Keypoint Identification 6

1) Attempt #1 6

2) Attempt #2 7

3) Attempt #3 8

5) Attempt #5 13

Problems Encountered 14

Future Work 15

Work in Progress 15

Changes Made 15

Conclusion 15

Introduction

Background Information

Robotic manipulation of soft objects poses a complex challenge in the field of robotics, requiring innovative solutions to address the uncertainties and dynamic nature of such materials. Deformable linear objects, such as cables, threads, or hoses, are present in various industries. For example, wires are manipulated to assemble devices in the Computer, Communication and Consumer Electronics industry, belts are used on assembly lines and sutures are meant to be intricately handled in the medical field. Conventional rigid-body manipulation techniques often fall short when applied to these flexible structures, requiring the development of advanced methodologies for efficient and precise manipulation.

The traditional robotic manipulation model, designed primarily for rigid objects, encounters difficulties in handling deformable linear objects due to factors such as elasticity and unpredictable behavior. Overcoming these challenges requires a paradigm shift in robotic manipulation strategies, integrating cutting-edge technologies to ensure adaptability, robustness, and enhanced manipulation capabilities. Which is exactly where this project comes in.

Purpose

The purpose of this project is to explore and develop a feasible approach to the robotic manipulation of soft objects, with a focus on deformable linear objects (DLOs), through the application of Graph Neural Networks (GNNs). GNNs have demonstrated remarkable success in capturing complex relationships within graph-structured data, making them particularly well-suited for modeling the dynamics exhibited by deformable structures. By leveraging the power of GNNs, I aim to enhance the adaptability and responsiveness of robotic systems, enabling them to manipulate deformable linear objects with a level of dexterity and precision previously unattainable through traditional methods.

This project seeks to contribute to the advancement of robotic manipulation capabilities, fostering innovation in automation technologies. Through the integration of GNNs, I aim to address the unique challenges posed by deformable linear objects.

Scope

The scope of this project encompasses the implementation of a robust robotic manipulation system capable of manipulating deformable linear objects. The research will involve the design and training of Graph Neural Networks tailored to the specific challenges presented by the nature of these objects. The evaluation of the system's performance will be conducted through a series of controlled experiments tests on accuracy and precision.

Objectives

1) Database creation

To facilitate the training of our Graph Neural Network (GNN) model, we plan to compile a dataset consisting of fifty 10-second videos, resulting in approximately 1,000 frames of training images. Each video captures a distinct movement of the training wire, strategically designed to cover a wide range of movements. By structuring the data in the form of distinct videos, we aim to provide the GNN with a nuanced perspective on various wire manipulations. This approach enables the model to discern patterns and correlations between successive frames, fostering the ability to predict future movements of the wire based on a history of observed images.

2) Image Preprocessing

Image processing involves cleaning up image noise and modelling the wire using nodes and edges to replicate a graph that can be processed by the GNN. The node data should include X and Y coordinates for the GNN to use.

3) GNN Model Development

Develop a robust Graph Neural Network model for predicting the movements of the training wire.

4) Training and Validation

Conduct extensive training and validation sessions for the GNN model using the compiled dataset. Fine-tune the model to accurately capture the dynamics of wire movements and ensure its adaptability across various scenarios. Ideally, the GNN should proficiently generate detailed edge relationship information between the constructed nodes.

5) Evaluation and Validation

Assess and compare the model's precision in predicting the dynamic behavior of the wire and its proficiency in accurately simulating the interconnections, or "edges," between the nodes of the wire.

6) Optimization for Real-time Inference:

Optimize the trained GNN model for real-time inference, considering computational efficiency and latency constraints. This step is crucial for ensuring the practical applicability of the model in robotics.

7) Robot Interaction Testing:

Explore the interaction between the GNN model and the robot system. Assess the system's ability to collaborate effectively with GNN output and adjust wire manipulations accordingly.

Timeline

Month	Week	Activity
Dec	1	Finalise Key point detection script and
	2	Consolidate Database and Image processing
	3	Feature Extraction in the form of matrices (derive X, Y data from each node)
	4	Feature Extraction in the form of matrices (derive X, Y data from each node)
Jan	1	GNN Model Development
	2	GNN Model Development
	3	Training the Model
	4	Training the Model
Feb	1	Evaluation and Validation
	2	Optimisation for Real Time Inference
	3	Robot interaction Testing
	4	Robot interaction Testing
Mar	1	Fine Tuning
	2	Fine Tuning (Start on draft Report)
	3	Fine Tuning
	4	Fine Tuning

Purpose of Report

The purpose of this report is to provide a brief description of the progress made for the project thus far, and the next steps to be taken in the near future.

Work Conducted

In summary, the following activities have been completed so far:

1) Research on DLO Tracking
2) Research on GNN
3) Image Processing method finalisation
4) Key point Identification

The following is a high-level description of the approach taken for the key point identification and image processing.

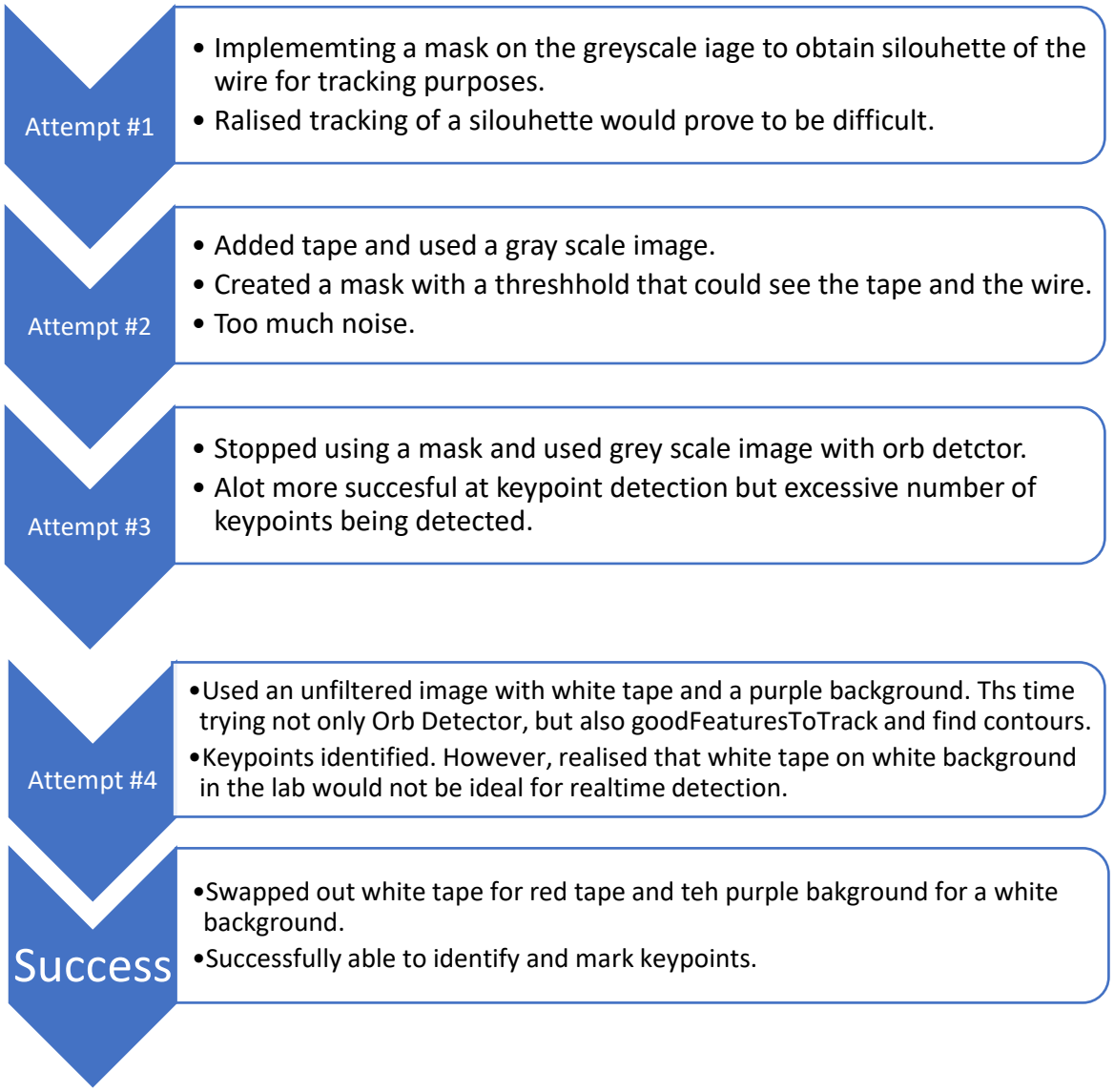

Image Processing and Keypoint Identification

1) Attempt #1

Initially, as I was still grasping computer vision concepts, I assumed that I would need to clean up an image of any form of noise and leave the key focus in the image before I could even start working on it. This led me to use a 2D median filter, 'medfilt2d', and a threshold, as seen in Figure 1.1. The function created a binary mask such that if a pixel in the original image is less than the corresponding pixel in the filtered mage minus 70, it would return True, and False otherwise. This resulted in the image seen in Figure 1.2.

```
bg = signal.medfilt2d(img, 11)
mask1 = img < bg - 70
mask1 = np.where(mask1, img, 0.0)
```

Figure1.1 Background Removal Function

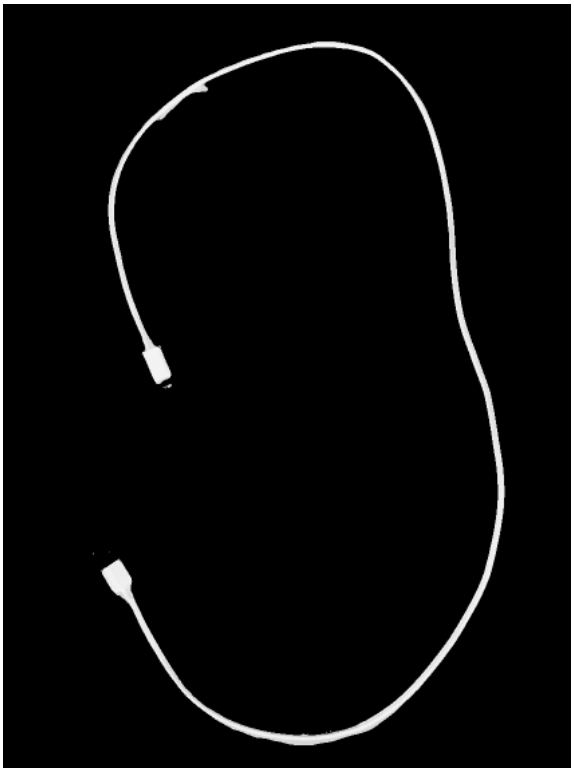

Figure 1.2

However, after some reading and research, I realized that this image although clean, would not help me identify the necessary keypoints for my GNN and that if there was a slight change in the conditions of the picture, this same function would not work. That's when I realised that I would probably need to use some identifiers for keypoints and that I would probably need to use a keypoint algorithm.

2) Attempt #2

This time around, I added white tape to the black wire and use a pink background. I continued using a greyscale image for processing and used a median mask to clean up he image, as seen in Figure 2.

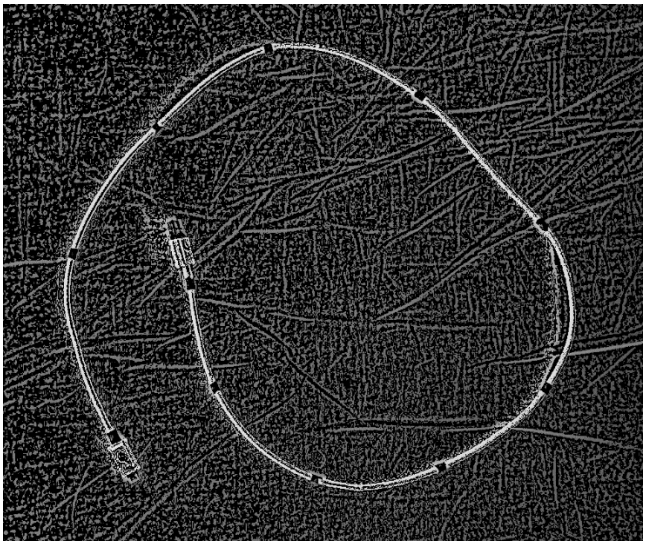

Figure 2

Although now I could see the key point identifiers clearly, there was still a lot of noise in the background. I then went on to research on key point identifier algorithms and realised that that may be a better option for this scenario to help me mark out the keypoints.

3) Attempt #3

The first key point algorithm I came across, was cv2's Orb algorithm. It was implemented using the code seen in Figure 3.1. I used the algorithm on a greyscale image of the wire which provided the result as seen in Figure 3.2.

```
def orb_detector(img):
    # Initialize the ORB detector
    orb = cv2.ORB_create()
    # Detect keypoints
    keypoints = orb.detect(img, None)
    # Draw keypoints on the image
    return cv2.drawKeypoints(img, keypoints, None, color=(0,255,0), flags=0)
```

Figure 3.1

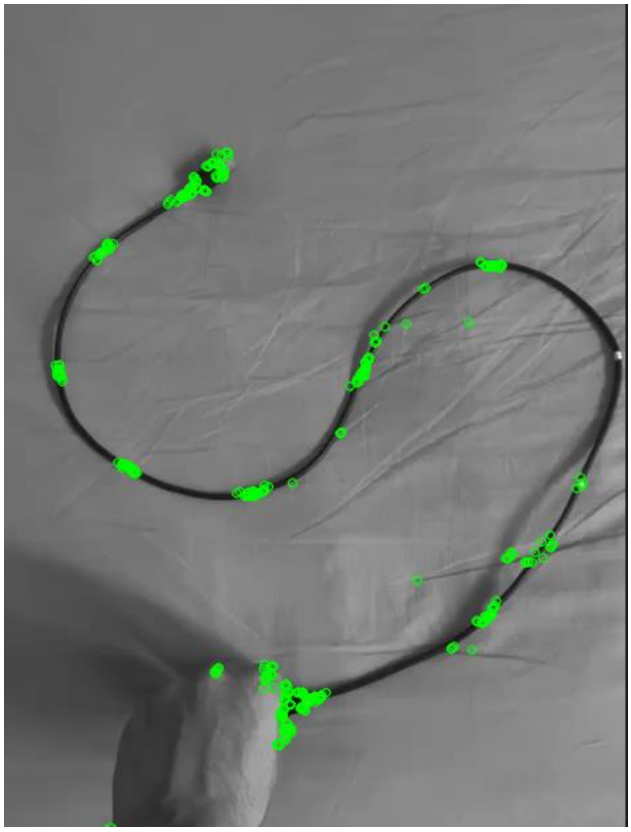

Figure 3.2

Although successful in identifying key points, I identified the same key point multiple times, which may not be ideal for modelling the wire. I tried other methods such as the 'goodFeaturesToTrack' algorithm and eventually created a function for contour tracking which turned out to work best, as explained in attempt #4.

4) Attempt #4

This time, I started off by using the Orb detector algorithm on an unfiltered image. Using the same function as in Figure 3.1 resulted in the following keypoints being detected, as seem in Figure 4.1.

Figure 4.1

Although the results were much better than when using a greyscale image, it was still identifying the keypoints multiple times. This led me to use the 'goodFeaturesToTrack' algorithm, which identifies corners of he keypoints instead of the whole keypoint, which I thought would be more beneficial. I implemented the algorithm using the function seen in Figure 4.2 which provided the result in Figure 4.3.

```
def goodFeaturesToTrack(img):
    # Convert BGR image to GRAY
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # corner detection algorithm
    corners = cv2.goodFeaturesToTrack(gray, 15, 0.01, 10) #paramters: max number of corners, corner quality level, minimum euclidean distance
    corners = np.intp(corners)
    copied_img = img.copy()
    for i in corners:
        x,y = i.ravel()
        cv2.circle(copied_img,(x,y),5,255,-1)
    return copied_img
```

Figure 4.2

Figure 4.3

Although minimizing the number of keypoints found, they were still not as accurate as I would have liked them to Be and some keypoints had even gone unnoticed. This is when I came up with the idea of using a contour detector instead and to pick the largest contours and to see the result that would be produced. I used a HSV image doe this case as I had read that HSV color space is often used in color-based image processing tasks because it separates color information from lighting/shading information. I implemented this idea using the following function after some rounds of testing as seen in Figure 4.4. I created a binary mask specifically for the white tape using the upper and lower thresholds of the HSV range of white. I then used the “findContours” function with RETR_TREE as my contour retrieval mode and CHAIN_APPROX_SIMPLE as the contour approximation method. This allowed me to extract Numpy arrays of (x, y) coordinates of boundary points of each contour. This data was then used in the next line to create a list of tuples that contained each contour and its corresponding area.

```python
# Function to check if two contours are close
def are_contours_close(cnt1, cnt2, threshold=10):
    x1, y1, w1, h1 = cv2.boundingRect(cnt1)
    x2, y2, w2, h2 = cv2.boundingRect(cnt2)

    # Calculate the distance between the centers of the bounding rectangles
    distance = ((x1 + w1 / 2 - x2 - w2 / 2) ** 2 + (y1 + h1 / 2 - y2 - h2 / 2) ** 2) ** 0.5
    return distance < threshold

def colorContour(img):
    # Convert BGR image to HSV
    hsv_image = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    # Define the range for white color
    lower_white = np.array([0, 0, 200])
    upper_white = np.array([180, 50, 255])
    # Create a mask
    mask = cv2.inRange(hsv_image, lower_white, upper_white)
    # Find contours
    contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    # Calculate area for each contour and create a tuple (contour, area)
    contour_areas = [(cnt, cv2.contourArea(cnt)) for cnt in contours]

    # Sort contours based on area in descending order
    contour_areas.sort(key=lambda x: x[1], reverse=True)

    # Select top 20 contours
    top_area_contours = [cnt[0] for cnt in contour_areas[:20]]

    # Filter out smaller contours that are close to larger ones
    filtered_contours = []
    for i, cnt1 in enumerate(top_area_contours):
        keep = True
        for cnt2 in top_area_contours:
            if cnt2 is not cnt1:
                if are_contours_close(cnt1, cnt2) and cv2.contourArea(cnt1) < cv2.contourArea(cnt2):
                    keep = False
                    break
        if keep:
            filtered_contours.append(cnt1)

    print("Number of fitlered contours", len(filtered_contours))
```

Figure 4.4

Before I include the loop to filter out smaller contours, I had just implemented this function with the idea of extracting the top 11 largest contour areas, instead of the current 20, as that was then number of keypoint identifiers I had on the wire. However, during testing I had noticed that in some angles, the same keypoint would be identified twice, and resultantly, one other keypoint would go unnoticed. That caused me to add the loop that would take into account proximity of the contours by using the 'boundingRect' information that can be extracted from the contours. I followed this by adding a bounding box around the keypoints to act as a node, using the following code in Figure 4.5.

```
# Draw bounding box around color block
copied_img = img.copy()
for contour in filtered_contours:
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(copied_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
    # Draw a cross marker
    point = (int(x+w/2), int(y+h/2))
    cv2.drawMarker(copied_img, point, color=(0, 0, 255), markerType=cv2.MARKER_CROSS, markerSize=20, thickness=2)

return copied_img
```

Figure 4.5

The above resulted in the following keypoints being detected, as seen in Figure 4.6.

Figure 4.6

I was quite happy with the results I had obtained in this case but realised that my threshold values of “lower_white” and “upper_white” would not be applicable in a environment with a white background like in the lab.

5) Attempt #5

This is how I came up with my final function, swapping out the white tape for red tape and replacing the threshold values to a HSV range for red. This resulted in the following change in the code, as seen in Figure 5.1 and the following result, in Figure 5.2.

```
lower_red = np.array([160, 50, 50])
upper_red = np.array([180, 255, 255])
# Create a mask
mask = cv2.inRange(hsv_image, lower_red, upper_red)
```

Figure 5.1

Figure 5.2

This to me was a successful keypoint identification.

Problems Encountered

In the course of the project, a significant challenge surfaced due to a gap in my initial understanding of Graph Neural Networks (GNNs). This knowledge gap led to a considerable amount of time being dedicated to image preprocessing for an unintended purpose. However, a good bit of research and learning, I knew exactly what needed to be done and was able to get back on track.

Choosing the right materials for the project turned out to be a bit tricky. Some tapes were too reflective, causing unwanted white spots in the images due to light reflections. This made it hard for me to process the images properly. After trying different options, I found that red duct tape worked the best. It didn't cause reflection issues and made the image processing much smoother.

Future Work

Work in Progress

I am currently working on consolidating my database.

Changes Made

The project's original requirements have remained largely unchanged. However, a subtle adjustment is worth noting: the project's focus has narrowed down to Deformable Linear Objects (DLOs), a subset within the broader category of soft objects. Despite this refinement, the overarching goal remains consistent — to address the challenges associated with soft object manipulation. Time permitting, there is a keen interest in exploring solutions for the broader issue of manipulating soft objects beyond the specific scope of DLOs.

Conclusion

The project is progressing quite smoothly, with keypoints being identified successfully and initial knowledge foundation being obtained. Juggling this project and my other school commitments was something I found quite challenging but I am quite pleased with the progress I have made and hope to be more effcient in the months to come.